

Toronto Survival Jobs:

Technical Project Documentation

- **Project Type:** ASP.NET Core MVC Web Application
 - **Database:** Supabase PostgreSQL
 - **Architecture:** MVC + Service Layer
 - **ORM:** Entity Framework Core
 - **HTML Parsing:** HtmlAgilityPack
 - **Frontend:** Razor Views + Tailwind CSS
 - **Primary Purpose:** Help users discover and track survival/employment opportunities in the Toronto area.
-

1. Project Overview

1.1 Project Description

Toronto Survival Jobs is an ASP.NET Core MVC web application designed to help users find and manage short-term, entry-level, part-time, full-time, and other employment opportunities in the Toronto area.

The application provides a centralized database of job listings and allows users to:

- Browse available jobs
- Search jobs by keyword
- Search jobs by location
- View detailed job information
- Manage companies
- Manage categories
- Manage locations
- Store salary information
- Track job sources
- Track when jobs were scraped
- Import jobs from scraped HTML content
- Associate jobs with companies and locations

The project also demonstrates database relationships and basic web scraping functionality.

2. Project Goals

The main goals of the project are:

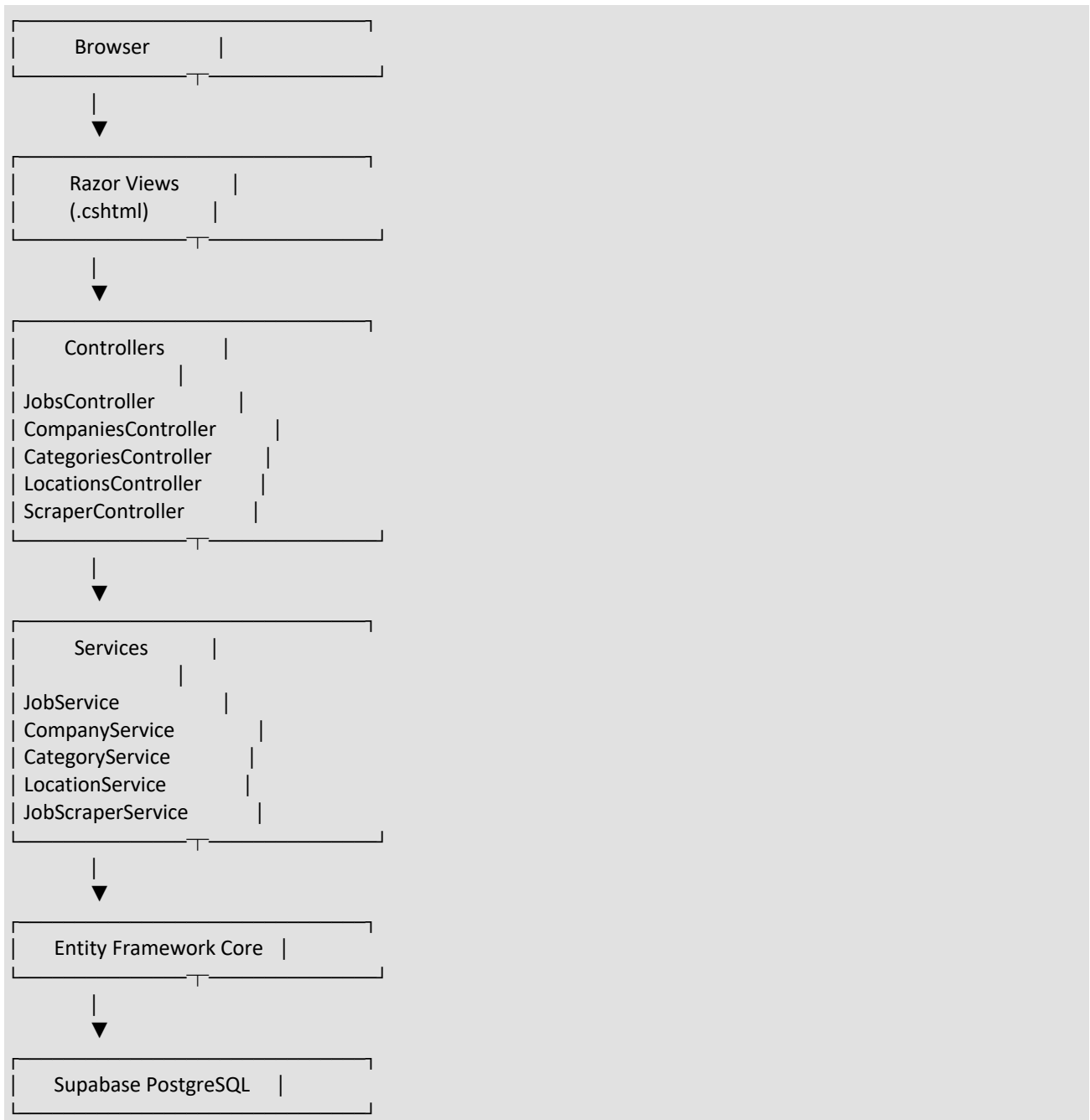
1. Build a practical ASP.NET Core MVC application.
 2. Demonstrate database connectivity using Supabase.
 3. Implement CRUD operations.
 4. Demonstrate relational database design.
 5. Implement a service layer.
 6. Implement job searching.
 7. Demonstrate HTML content scraping.
 8. Automatically associate scraped jobs with existing or new companies.
 9. Prevent duplicate job records.
 10. Provide a simple and user-friendly interface.
-

3. Technologies Used

Technology	Purpose
ASP.NET Core MVC	Web application framework
C#	Primary programming language
Entity Framework Core	Database access and ORM
Supabase	PostgreSQL database hosting
PostgreSQL	Relational database
Razor Views	Server-side UI
Tailwind CSS	UI styling
HtmlAgilityPack	HTML parsing and scraping
LINQ	Database querying
Dependency Injection	Service management
Git/GitHub	Version control

4. Application Architecture

The application follows a simplified MVC architecture with a Service Layer.



5. Project Structure

The project follows this general structure:

```
TorontoSurvivalJobs/
|
|-- Controllers/
|   |-- JobsController.cs
|   |-- CompaniesController.cs
|   |-- CategoriesController.cs
```

```
|   |-- LocationsController.cs
|   |-- ScraperController.cs
|
|   |-- Models/
|   |   |-- Job.cs
|   |   |-- Company.cs
|   |   |-- Category.cs
|   |   |-- Location.cs
|   |   |-- ScrapedJob.cs
|   |
|   |-- Services/
|   |   |-- JobService.cs
|   |   |-- CompanyService.cs
|   |   |-- CategoryService.cs
|   |   |-- LocationService.cs
|   |   |-- JobScraperService.cs
|   |
|   |-- Data/
|   |   |-- ApplicationDbContext.cs
|   |
|   |-- Views/
|   |   |-- Jobs/
|   |   |   |-- Index.cshtml
|   |   |   |-- Details.cshtml
|   |   |   |-- Create.cshtml
|   |   |   |-- Edit.cshtml
|   |   |   |-- Delete.cshtml
|   |   |   |-- Search.cshtml
|   |   |
|   |   |-- Companies/
|   |   |   |-- Index.cshtml
|   |   |   |-- Details.cshtml
|   |   |   |-- Create.cshtml
|   |   |   |-- Edit.cshtml
|   |   |   |-- Delete.cshtml
|   |   |
|   |   |-- Categories/
|   |   |   |-- Index.cshtml
|   |   |   |-- Details.cshtml
|   |   |   |-- Create.cshtml
|   |   |   |-- Edit.cshtml
|   |   |   |-- Delete.cshtml
|   |   |
|   |   |-- Locations/
|   |   |   |-- Index.cshtml
|   |   |   |-- Details.cshtml
|   |   |   |-- Create.cshtml
|   |   |   |-- Edit.cshtml
|   |   |   |-- Delete.cshtml
|   |   |
|   |   |-- Scraper/
|   |   |   |-- Index.cshtml
|   |   |   |-- Results.cshtml
|   |
|   |-- wwwroot/
|   |   |-- css/
|   |   |-- js/
```

```
| └─ sample-jobs/
|   └─ toronto-jobs.html
|
| └─ Program.cs
└─ appsettings.json
```

6. Database Design

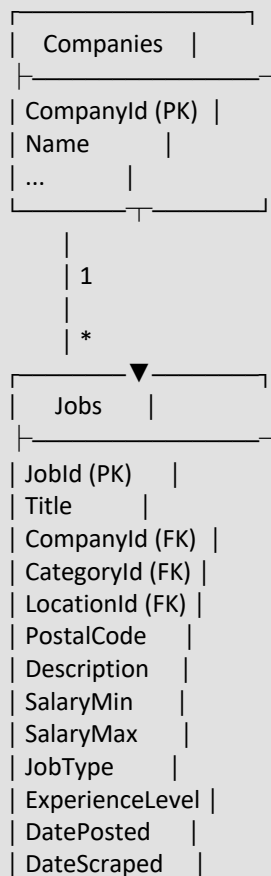
The application uses a relational database hosted on Supabase.

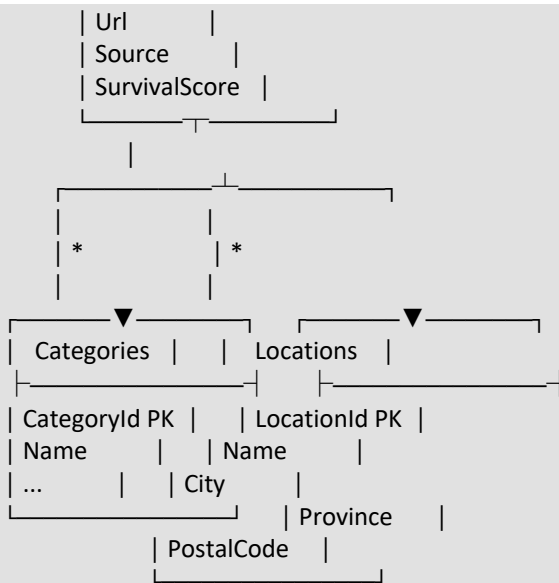
6.1 Main Tables

Table	Purpose
Jobs	Stores job listings
Companies	Stores employer/company information
Categories	Groups jobs into categories
Locations	Stores geographic information

7. Entity Relationship Diagram

The database relationship can be represented as:





8. Table Relationships

8.1 Company → Jobs

- **Relationship:** One-to-Many
- One company can have many jobs.

The Jobs table contains CompanyId as a foreign key. In the Job model:

```
public int CompanyId { get; set; }
public Company? Company { get; set; }
```

8.2 Category → Jobs

- **Relationship:** One-to-Many
- A category can contain many jobs.

The Job model contains:

```
public int? CategoryId { get; set; }
public Category? Category { get; set; }
```

Note: The relationship is optional because a job may not necessarily have a category assigned.

8.3 Location → Jobs

- **Relationship:** One-to-Many
- A location can have multiple jobs.

The Job model contains:

```
public int? LocationId { get; set; }
public Location? Location { get; set; }
```

This avoids duplicated location information by linking Job.LocationId $\rightarrow$ Location.LocationId $\rightarrow$ Location.Name.

9. Model Specifications

9.1 Job Model

```
using System.ComponentModel.DataAnnotations;
```

```
namespace TorontoSurvivalJobs.Models
{
    public class Job
    {
        public int JobId { get; set; }

        [Required]
        public string Title { get; set; } = string.Empty;

        [Required]
        public string? PostalCode { get; set; }

        public string? Description { get; set; }

        public decimal? SalaryMin { get; set; }

        public decimal? SalaryMax { get; set; }

        public string? JobType { get; set; }

        public string? ExperienceLevel { get; set; }

        public DateOnly? DatePosted { get; set; }

        public DateTime DateScraped { get; set; }

        public string? Url { get; set; }

        public string? Source { get; set; }

        public int SurvivalScore { get; set; }

        public int? CategoryId { get; set; }

        public Category? Category { get; set; }

        public int CompanyId { get; set; }

        public Company? Company { get; set; }

        public int? LocationId { get; set; }

        public Location? Location { get; set; }
    }
}
```

Job Fields

Property	Type	Description
JobId	int	Primary key
Title	string	Job title
PostalCode	string	Job postal code
Description	string	Job description
SalaryMin	decimal?	Minimum salary
SalaryMax	decimal?	Maximum salary
JobType	string	Full-time, part-time, etc.
ExperienceLevel	string	Required experience
DatePosted	DateOnly?	Date job was posted
DateScraped	DateTime	Date job was collected
Url	string	Original job URL
Source	string	Source website
SurvivalScore	int	Job usefulness score
CompanyId	int	Company foreign key
CategoryId	int?	Category foreign key
LocationId	int?	Location foreign key

9.2 Company Model

```
public class Company
{
    public int CompanyId { get; set; }

    public string Name { get; set; } = string.Empty;

    public ICollection<Job> Jobs { get; set; } = new List<Job>();
}
```

9.3 Category Model

```
public class Category
{
    public int CategoryId { get; set; }

    public string Name { get; set; } = string.Empty;

    public ICollection<Job> Jobs { get; set; } = new List<Job>();
}
```

Example categories: Warehouse, Cleaning, Restaurant, Retail, Construction, Delivery, General Labour.

9.4 Location Model

```
public class Location
{
    public int LocationId { get; set; }

    public string Name { get; set; } = string.Empty;

    public string? City { get; set; }

    public string? Province { get; set; }

    public string? PostalCode { get; set; }

    public ICollection<Job> Jobs { get; set; } = new List<Job>();
}
```

9.5 ScrapedJob Model

ScrapedJob is not a database entity. It represents temporary data extracted by the scraper to decouple scraping logic from database entities.

```
public class ScrapedJob
{
    public string Title { get; set; } = string.Empty;

    public string CompanyName { get; set; } = string.Empty;

    public string? LocationName { get; set; }

    public decimal? SalaryMin { get; set; }

    public decimal? SalaryMax { get; set; }

    public string? JobType { get; set; }

    public string? Description { get; set; }

    public string? Url { get; set; }

    public string? Source { get; set; }

    public DateTime DateScraped { get; set; }
}
```

10. Controllers

Controllers handle HTTP requests and coordinate between views and services.

10.1 JobsController

Responsible for listing, viewing, creating, editing, deleting, and searching jobs.

Typical actions: Index(), Details(), Create(), Edit(), Delete(), Search()

```
[HttpGet]
public async Task<ActionResult> Search(string? query, string? location)
{
    var jobs = await _jobService.SearchJobs(query, location);

    ViewBag.Query = query;
    ViewBag.Location = location;

    return View(jobs);
}
```

10.2 CompaniesController

Responsible for company CRUD operations (Index, Details, Create, Edit, Delete). The details view displays all jobs associated with that company.

10.3 CategoriesController

Responsible for managing categories and grouping jobs via CategoryId.

10.4 LocationsController

Responsible for managing locations to allow key reuse across multiple job listings.

10.5 ScraperController

Manages the web scraping workflow (Index(), Scrape(), Save()).

```
User opens /Scraper
↓
Clicks Scrape Jobs
↓
ScraperController
↓
JobScraperService
↓
ScrapedJob objects
↓
Results.cshtml
↓
User clicks Save
↓
CompanyService / LocationService / JobService
↓
Database
```

11. Service Layer

The application utilizes a dedicated service layer to separate business logic from controller actions.

```
Services/
├─ JobService.cs
├─ CompanyService.cs
├─ CategoryService.cs
```

11.1 JobService Example

```
public async Task<List<Job>> SearchJobs(string? query, string? location)
{
    var jobs = _context.Jobs
        .Include(j => j.Company)
        .Include(j => j.Category)
        .Include(j => j.Location)
        .AsQueryable();

    if (!string.IsNullOrEmpty(query))
    {
        query = query.Trim();

        jobs = jobs.Where(j =>
            j.Title.Contains(query) ||
            (j.Description != null && j.Description.Contains(query)));
    }

    if (!string.IsNullOrEmpty(location))
    {
        location = location.Trim();

        jobs = jobs.Where(j =>
            j.Location != null && j.Location.Name.Contains(location));
    }

    return await jobs
        .OrderByDescending(j => j.SurvivalScore)
        .ToListAsync();
}
```

11.2 CompanyService Workflow

Handles getting or creating companies dynamically during job import/scraping:

```
public async Task<Company> GetOrCreateCompany(string name)
{
    var company = await GetCompanyByName(name);
    if (company != null)
    {
        return company;
    }

    company = new Company { Name = name };
    await CreateCompany(company);
    return company;
}
```

11.3 LocationService Example

```
public async Task<Location> GetOrCreateLocation(string name)
{
    var location = await GetLocationByName(name);
```

```

if (location != null)
{
    return location;
}

location = new Location
{
    Name = name
};

await CreateLocation(location);

return location;
}

```

12. Web Scraping & HTML Parsing

The JobScraperService extracts job postings from raw HTML using **HtmlAgilityPack**.

```

var document = new HtmlDocument();
document.LoadHtml(html);

var jobCards = document.DocumentNode.SelectNodes("//div[contains(@class, 'job-card')]");

```

HTML Source Field Extraction

Field	HTML Source
Title	.job-title
Company	.company
Location	.location
Salary	.salary
Job Type	.job-type
Description	.description
URL	

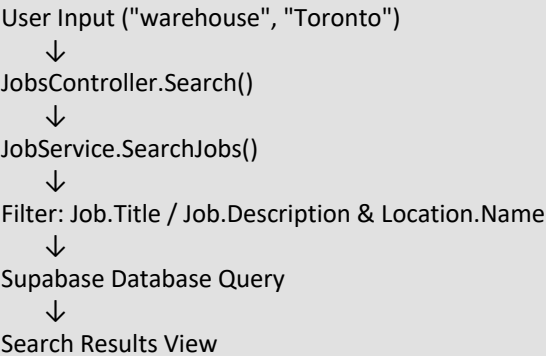
Salary Parsing Strategy

- **Range:** "\$18 - \$22/hour" $\rightarrow$ SalaryMin = 18, SalaryMax = 22
- **Single Value:** "\$18/hour" $\rightarrow$ SalaryMin = 18, SalaryMax = 18

13. Search Implementation

Search functionality provides a two-field interface (**WHAT** and **WHERE**).

```
<form method="get" asp-controller="Jobs" asp-action="Search">
  <!-- WHAT: Title / Keyword -->
  <!-- WHERE: Remote or City -->
  <button type="submit">Search</button>
</form>
```



14. Views & UI Layer

Built with server-rendered **Razor Views** and **Tailwind CSS** for responsive layout design.

Directory Structure

- Views/Jobs/: Index.cshtml, Details.cshtml, Create.cshtml, Edit.cshtml, Delete.cshtml, Search.cshtml
- Views/Companies/: CRUD views for managing companies.
- Views/Categories/: CRUD views for category taxonomy.
- Views/Locations/: CRUD views for geographical regions.
- Views/Scraper/: Index.cshtml, Results.cshtml

15. Core Application Features & Implementation Status

Feature	Status
Job CRUD	Implemented
Company CRUD	Implemented
Category CRUD	Implemented
Location Relationship	Implemented
Company \$	
rightarrow\$ Jobs Relationship	Implemented
Job Details View	Implemented
Job Keyword Search	Implemented

Feature	Status
Location-based Search	Implemented
Salary Parsing Logic	Implemented
HTML Scraping	Implemented
Scraped Job Preview	Implemented
Scraped Job Saving	Implemented
Duplicate Detection	Implemented
External Job Website Scraping	Future Improvement
Advanced Dynamic Filters	Future Improvement

16. Security, Validation, and Architecture Best Practices

- 1. Model Validation:** Data Annotations (e.g., [Required]) are enforced prior to database persistence.
- 2. Configuration & Secrets:** Database strings and API keys are strictly managed via appsettings.json, Environment Variables, or User Secrets.
- 3. Scraping Compliance:** Scraper components adhere to web scraping guidelines (robots.txt, request throttling, TOS compliance).
- 4. Duplicate Prevention:** Before creating a record from scraping, existing entries are checked using JobExists(title, companyId) logic.
- 5. Dependency Injection:** Services (JobService, CompanyService, etc.) are registered as Scoped dependencies in Program.cs:

```
builder.Services.AddScoped<JobService>();
builder.Services.AddScoped<CompanyService>();
builder.Services.AddScoped<CategoryService>();
builder.Services.AddScoped<LocationService>();
builder.Services.AddScoped<JobScraperService>();
```

17. Conclusion

Toronto Survival Jobs demonstrates a complete, normalized, database-driven ASP.NET Core MVC architecture. By decoupling business logic into a Service Layer and normalizing related models (Company, Location, Category), the application achieves high maintainability and performance while delivering practical tools for job seekers in the Toronto area.